

Esame Programmazione di Sistema / API Programming

Data: 2024-06-25

QUESTIONS

1 Teoria #1 (3 pt)

Si consideri il programma seguente che riporta la numerazione delle linee di codice.

```
1 fn main() {  
2     let numbers = vec![1,2,3,4,5,8];  
3  
4     let res = numbers  
5         .iter()  
6         .filter(|&x| x % 2 == 0)  
7         .zip('a'..'z');  
8  
9     let last = res  
10        .clone()  
11        .map(|(a,b)|{format!("{b}{a}")})  
12        .last();  
13  
14    println!("last: {:?}", last);  
15    println!("res: {:?}", res.count());  
16 }
```

- Che cosa stampa questo codice?
- Che cosa fanno le istruzioni alle righe 5, 6, 7?
- Che cosa capita se si omette la riga 10 (`.clone()`)? Perché?

2 Teoria #2 (3 pt)

Si descriva il comportamento di questo programma. Se presenta delle problematiche, come può essere modificato?

```
1 use std::sync::{Arc, Mutex, Condvar};  
2 use std::thread;  
3 use std::time::Duration;  
4  
5 fn main() {  
6     let pair = Arc::new((Mutex::new(false), Condvar::new()));  
7     let pair2 = Arc::clone(&pair);  
8  
9     thread::spawn(move || {  
10         let (lock, cvar) = &*pair2;
```

```

11         let mut started = lock.lock().unwrap();
12         *started = true;
13         cvar.notify_one();
14     });
15
16     let (lock, cvar) = &*pair;
17     println!("Waiting ...");
18     thread::sleep(Duration::from_secs(1));
19
20     let mut started = lock.lock().unwrap();
21     started = cvar.wait(started).unwrap();
22     println!("End!");
23 }

```

3 Teoria #3 (3 pt)

Il codice seguente genera un errore di compilazione: spiegare perché e indicare come modificare la `struct S` (attraverso l'aggiunta di tratti) per renderlo compilabile ed eseguibile.

```

1 #[derive(Debug)]
2 struct S {
3     i: i32,
4 }
5
6 impl From<i32> for S {
7     fn from(value: i32) -> Self {
8         S { i: value }
9     }
10 }
11
12 fn main() {
13     let mut v = Vec::<S>::new();
14     let s = 42.into();
15
16     for _i in 0..3 {
17         v.push(s);
18     }
19     println!("{:?}", v);
20 }

```

4 Programmazione (6 pt)

Si realizzi l'implementazione della struttura dati `Exchanger<T: Send>` (e dei metodi e delle funzioni necessarie) utile per realizzare una comunicazione bidirezionale.

Ciascun lato della comunicazione dispone di un'istanza della struttura `Exchanger<T: Send>`.

La comunicazione avviene invocando il metodo

```

1 fn exchange(&self, t: T) -> Option<T>

```

che, una volta invocato, si blocca fino a quando non viene invocato il metodo corrispondente sulla struttura corrispondente al lato opposto della comunicazione, dopodiché restituisce il valore che è stato passato come argomento al metodo corrispondente al lato opposto (che farà altrettanto), sotto forma di `Some(t)`.

Lo scambio può essere ripetuto un numero arbitrario di volte.

Se una delle due strutture formanti la coppia viene distrutta, un'eventuale chiamata, bloccata sul metodo della struttura restante, terminerà restituendo il valore `None`.

Si implementi tale struttura in linguaggio Rust avendo cura che la sua implementazione sia *thread-safe*.

SOLUTIONS

1 Teoria #1

Che cosa stampa questo codice?

Il codice stampa:

```
last: Some("c8")
res: 3
```

Spiegazione: l'iteratore `res` contiene le tuple composte dai numeri pari filtrati e le lettere dell'alfabeto, ovvero `(2, 'a')`, `(4, 'b')`, `(8, 'c')`. La variabile `last` mappa questi elementi in stringhe scambiando l'ordine (es. `"a2"`) e prende l'ultimo elemento, che risulta essere `Some("c8")`. La variabile `res` viene poi consumata da `count()`, che conta il numero di elementi (3).

Che cosa fanno le istruzioni alle righe 5, 6, 7?

Queste istruzioni creano una catena di iteratori a partire dal vettore `numbers`:

- `.iter()` crea un iteratore che produce reference agli elementi del vettore.
- `.filter(|&x| x % 2 == 0)` filtra gli elementi mantenendo solo i numeri pari.
- `.zip('a'..'z')` accoppia (zippa) ciascun numero pari con una lettera dell'alfabeto, producendo un iteratore di tuple `(&i32, char)`.

Che cosa capita se si omette la riga 10 (`.clone()`)? Perché?

Se la chiamata a `.clone()` viene omessa, il programma genera un errore di compilazione (`use of moved value: res`). Questo accade perché i metodi `map` e `last` consumano l'iteratore su cui sono chiamati. Senza clonarlo, l'iteratore `res` viene mosso e consumato nel blocco che definisce `last`, e pertanto non risulta più disponibile alla riga 15 per chiamare `res.count()`.

2 Teoria #2

Comportamento del programma e problematiche:

Il programma soffre del classico problema del *lost wakeup* (risveglio perso). Il thread principale attende 1 secondo prima di acquisire il lock e chiamare `wait()` sulla `Condvar`. Nel frattempo, il thread spawnato acquisisce immediatamente il lock, imposta `started` a `true` e chiama `notify_one()`. Poiché il thread principale è ancora in `sleep` e non sta aspettando sulla `Condvar`, la notifica viene persa. Quando il thread principale si risveglia dallo `sleep`, acquisisce il lock e chiama `wait()`, rimanendo bloccato all'infinito perché nessuno invierà più una notifica.

Inoltre, l'attesa sulla condizione `started` non avviene all'interno di un ciclo (loop), contravvenendo alle best practice per prevenire gli *spurious wakeups*.

Come modificarlo:

Per correggere il problema è necessario testare la condizione all'interno di un ciclo `while` nel main thread:

```
1 let mut started = lock.lock().unwrap();
2 while !*started {
3     started = cvar.wait(started).unwrap();
4 }
5 println!("End!");
```

In questo modo, poiché `started` è già `true` quando il main thread si sveglia dallo `sleep` di 1 secondo, il ciclo `while` termina immediatamente senza chiamare `wait()`, stampando "End!" correttamente.

3 Teoria #3

Perché genera errore:

Il compilatore restituisce un errore del tipo `use of moved value: s` alla riga in cui viene effettuato il `push`. Nel Rust, per impostazione predefinita, le `struct` hanno semantica di *move* (spostamento). Quando eseguiamo `v.push(s)` nella prima iterazione del ciclo, il valore di `s` viene spostato all'interno del vettore `v`. Alla seconda iterazione, si tenta di spostare nuovamente `s`, ma la variabile non possiede più il valore.

Come modificarlo:

Per fare in modo che l'inserimento non sposti (*move*) la variabile ma ne crei una copia, è necessario che la `struct S` implementi i tratti `Clone` e `Copy`. Questo si ottiene facilmente estendendo l'attributo `#[derive(...)]`:

```
1 #[derive(Debug, Clone, Copy)]
2 struct S {
3     i: i32,
4 }
```

In questo modo, ogni chiamata a `push` utilizzerà una copia della struttura, permettendo al codice di compilare ed essere eseguito correttamente.

4 Programmazione

Per implementare uno scambio bidirezionale e thread-safe, è possibile utilizzare un `Mutex` condiviso tramite `Arc` per proteggere lo stato della comunicazione, insieme ad una `Condvar` per bloccare i thread in attesa della controparte. Per distinguere i due lati della comunicazione (le due istanze di `Exchanger`), assegniamo un flag `is_left` (oppure un identificativo) a ciascuno.

```
1 use std::sync::{Arc, Mutex, Condvar};
2
3 enum State<T> {
4     Idle,
5     Offered { item: T, by_left: bool },
6     Exchanged(T),
7     Closed,
8 }
9
10 struct Inner<T> {
11     state: Mutex<State<T>>,
12     cvar: Condvar,
13 }
14
15 pub struct Exchanger<T: Send> {
16     inner: Arc<Inner<T>>,
17     is_left: bool,
18 }
19
20 pub fn new_exchanger<T: Send>() -> (Exchanger<T>, Exchanger<T>) {
21     let inner = Arc::new(Inner {
22         state: Mutex::new(State::Idle),
23         cvar: Condvar::new(),
24     });
25     (
26         Exchanger { inner: inner.clone(), is_left: true },
27         Exchanger { inner, is_left: false },
28     )
29 }
30
```

```

31 impl<T: Send> Exchanger<T> {
32     pub fn exchange(&self, t: T) -> Option<T> {
33         let mut state = self.inner.state.lock().unwrap();
34         let mut my_item = Some(t);
35
36         loop {
37             // Estraiamo lo stato attuale sostituendolo temporaneamente
con Idle
38             match std::mem::replace(&mut *state, State::Idle) {
39                 State::Closed => {
40                     *state = State::Closed;
41                     return None;
42                 }
43                 State::Idle => {
44                     // Nessuno ha offerto nulla. Offriamo il nostro
valore.
45                     *state = State::Offered {
46                         item: my_item.take().unwrap(),
47                         by_left: self.is_left,
48                     };
49                     loop {
50                         match std::mem::replace(&mut *state, State::
Idle) {
51                             State::Offered { item, by_left } => {
52                                 // Ancora in attesa. Rimettiamo lo
stato e aspettiamo.
53                                 *state = State::Offered { item, by_left
};
54                                 state = self.inner.cvar.wait(state).
unwrap();
55                             }
56                             State::Exchanged(other) => {
57                                 // L'altra parte ha preso il nostro e
ha lasciato il suo.
58                                 *state = State::Idle;
59                                 self.inner.cvar.notify_all();
60                                 return Some(other);
61                             }
62                             State::Closed => {
63                                 *state = State::Closed;
64                                 return None;
65                             }
66                             State::Idle => unreachable!(),
67                         }
68                     }
69                 }
70                 State::Offered { item, by_left } => {
71                     if by_left == self.is_left {
72                         // Un altro thread sul NOSTRO stesso lato ha
fatto un'offerta.
73                         // Rimettiamo lo stato e attendiamo.
74                         *state = State::Offered { item, by_left };
75                         state = self.inner.cvar.wait(state).unwrap();
76                     } else {
77                         // Il lato opposto ha fatto un'offerta!
Effettuiamo lo scambio.
78                         *state = State::Exchanged(my_item.take().unwrap
());

```

```

79         self.inner.cvar.notify_all();
80         return Some(item);
81     }
82 }
83 State::Exchanged(other) => {
84     // C'e' un elemento scambiato non ancora prelevato.
85     Attendiamo.
86     *state = State::Exchanged(other);
87     state = self.inner.cvar.wait(state).unwrap();
88 }
89 }
90 }
91 }
92
93 impl<T: Send> Drop for Exchanger<T> {
94     fn drop(&mut self) {
95         let mut state = self.inner.state.lock().unwrap();
96         *state = State::Closed;
97         self.inner.cvar.notify_all();
98     }
99 }

```